

demo1 扫描画布结构说明

目的：说明`demo1`画布由 ACC 源码扫描生成后的层级结构、各模块功能、内部公式和决策逻辑。本文基于`project/demo1/data/cbdes.db`的画布结构，以及 GAASD 扫描缓存中的 ACC 源码逐层核对。

1. 结论

`demo1`是 GAASD 根据 ACC C++ 源码扫描生成的 Pangu/THICV 示例工程，不是手工拖拽基础模块搭建的旧版 ACC 画布。

当前画布能够和源码结构对应上：

画布层级	画布名称	对应源码类/函数	作用
应用层	空应用`app_empty`	GAASD 工程容器	承载进程和模块
进程层	空进程`process_empty`	GAASD 进程容器	当前未放业务逻辑
模块层	ACC参考模块`ACCModule`	Pangu`ACCModule`	对接`acc_input` / `acc_output` 通道
算法入口	ACC目标速度入口函数`	`control::AccTargetSpeed::run()`	调度决策、巡航设定、目标速度计算
子模块	ACC决策状态管理`	`AccDecisionStage::run()`	计算系统状态、决策编号、使能状态
子模块	ACC巡航设定值管理`	`AccCruiseSettingStage::run()`	管理时距和最大速度参数
子模块	ACC目标速度门控计算`	`AccTargetSpeedStage::run()`	计算目标速度并按 enable 门控输出

画布数据库位置：

```
/home/aiden/文档/Modularization/project/demo1/data/cbdes.db
```

源码缓存位置：

```
/home/aiden/gaasd_versions/gaasd-2.7.0.5/home/gaasd_server/components/THICV/extracted_c_functions/final_codescan/sources/acc_input_acc_target_speed_c395800a
```

2. 顶层模块结构

2.1 `ACCModule`

`ACCModule`是 Pangu 节点模块层，包含：

子项	类型	作用
`acc_input`	输入通道	订阅 ACC 所需输入消息
`acc_output`	输出通道	发布 ACC 计算结果
`run` / `ACC目标速度入口函数`	复合算法块	执行 ACC 决策、参数更新和目标速度计算

2.2 输入通道 `acc\_input`

输入通道对应 `pangu.modules.AccInput`：

字段	类型	含义
`frame_id`	`uint32`	帧编号，透传到输出
`ego_speed_mps`	`double`	自车速度，单位 m/s
`lead_speed_mps`	`double`	前车速度，单位 m/s
`lead_distance_m`	`double`	前车距离，单位 m
`command_type`	`int32`	驾驶指令类型，0 表示无指令

2.3 输出通道 `acc\_output`

输出通道对应 `pangu.modules.AccOutput`：

字段	类型	含义
`frame_id`	`uint32`	输入帧编号透传
`target_speed_mps`	`double`	ACC 输出目标速度，单位 m/s
`enable`	`bool`	ACC 对外控制使能
`valid`	`bool`	输出是否有效
`time_gap_s`	`double`	当前目标时距，单位 s
`max_speed_mps`	`double`	当前巡航速度上限，单位 m/s
`decision`	`int32`	当前决策编号
`system_state`	`int32`	当前系统状态编号

3. 算法入口 `AccTargetSpeed::run`

画布名称：`ACC目标速度入口函数`

源码功能：

```
void AccTargetSpeed::run(const Input& input, Output& output)
```

输入：

```
egoSpeed, leadSpeed, leadDistance, commandType
```

输出：

```
targetSpeed, enable, valid, timeGap, maxSpeed, decision, systemState
```

执行顺序：

1. 调用 `AccDecisionStage`，根据 `egoSpeed` 和 `commandType` 得到 `enable`、`valid`、`decision`、`systemState`。
2. 调用 `AccCruiseSettingStage`，根据 `decision` 更新 `timeGap` 和 `maxSpeed`。

3. 调用 `AccTargetSpeedStage`，根据自车/前车状态、`timeGap`、`maxSpeed` 和 `enable` 得到 `targetSpeed`。
4. 汇总所有结果输出到 `acc\_output`。

整体数据流：

```
acc_input
-> AccTargetSpeed
-> AccDecisionStage
-> AccCruiseSettingStage
-> AccTargetSpeedStage
-> acc_output
```

4. 子模块一：ACC 决策状态管理

画布名称：`ACC决策状态管理`

源码类：

AccDecisionStage

输入：

输入	含义
`egoSpeed`	自车速度
`commandType`	驾驶指令

输出：

输出	含义
`enable`	本周期对外 ACC 使能
`valid`	本周期输出是否有效
`decision`	决策编号
`systemState`	系统状态编号

内部状态：

状态	含义
`controlEnabled`	ACC 内部是否处于在控记忆状态
`hasHistory`	是否存在历史巡航状态，可用于恢复
`lastDecision`	上一次有效决策编号

内部子模块：

子模块	对应源码	功能
`ACC系统状态判定`	`DetermineAccSystemState`	根据速度、在控记忆、历史状态判定 S0~S3
`ACC决策编号判定`	`EvaluateAccDecision`	根据 `systemState` 和 `commandType` 输出 R1~R8 决策编号
`ACC使能状态判定`	`DetermineAccEnable`	计算本周期 enable 和下一周期 enable 记忆
`ACC上一次有效决策选择`	`SelectAccLastDecision`	保存 1~7 范围内的有效决策

4.1 `DetermineAccSystemState`：系统状态判定

功能：把当前 ACC 状态离散成系统状态编号。

输入：

egoSpeed, vMin, controlEnabled, hasHistory

逻辑：

```
if egoSpeed < vMin:
    systemState = 3
else if controlEnabled:
    systemState = 0
else if hasHistory:
    systemState = 1
else:
    systemState = 2
```

状态含义：

编号	含义
`S0 / 0`	正常在控状态
`S1 / 1`	非在控，但存在可恢复历史
`S2 / 2`	非在控，且无历史
`S3 / 3`	低速状态

作用：为后续决策表提供“现在 ACC 处于什么状态”的离散输入。

4.2 `EvaluateAccDecision`：决策编号判定

功能：把系统状态和驾驶指令映射成 ACC 决策编号。

输入：

systemState, commandType

主要映射：

systemState	commandType	decision	含义
0 在控	1	1	降低最大设定速度
0 在控	2	2	提高最大设定速度
0 在控	3	3	减小时距
0 在控	4	4	增大时距
0 在控	5	7	扭矩/油门相关决策占位
0 在控	6 或 7	8	取消或退出
1 有历史待命	1	5	重新设定/启控
1 有历史待命	2	6	恢复历史巡航
1 有历史待命	6 或 7	8	取消或退出
2 无历史待命	1	5	首次设定/启控
2 无历史待命	6 或 7	8	取消或退出
3 低速	任意	8	默认退出/无有效动作

说明：

- 当前源码中，没有匹配到具体动作时也输出 `decision = 8`。
- `decision = 8` 本身不直接更新时距或限速；真正取消/退出由 `DetermineAccEnable` 结合 `commandType == 6/7` 处理。

4.3 `DetermineAccEnable`：使能状态判定

功能：把瞬时驾驶指令转换为可持续的 ACC 在控记忆。

输入：

egoSpeed, vMin, controlEnabled, decision, commandType

核心逻辑：

```

speedReady = egoSpeed >= vMin
turnOn = (decision == 5) OR (decision == 6)
cancel = (commandType == 6) OR (commandType == 7)
keepOrTurnOn = controlEnabled OR turnOn

enableOutput = speedReady AND controlEnabled
enableNext = keepOrTurnOn AND NOT cancel

```

两个输出的区别：

输出	含义
`enableOutput`	本周期真正对外输出的 ACC 使能
`enableNext`	写入状态，供下一周期继续保持 ACC 在控

作用：

- 按一次启控指令后，`enableNext` 会被写入 `controlEnabled`。
- 下一周期即使 `commandType` 回到 0，也能继续保持在控。
- `commandType == 6/7` 会清除下一周期在控状态。

4.4 `SelectAccLastDecision`：上一次有效决策选择

功能：保存最近一次有效决策编号。

逻辑：

```
if 1 <= decision <= 7:  
    selectedDecision = decision  
else:  
    selectedDecision = lastDecision
```

作用：

- 只记忆 R1~R7 这类真实动作决策。
- `decision = 8` 代表取消/无动作，不覆盖历史有效决策。

4.5 `AccDecisionStage` 状态写回

`AccDecisionStage` 最后会更新内部状态：

```
controlEnabled = enableNext  
hasHistory = hasHistory OR enableNext  
lastDecision = selectedDecision
```

作用：

- `controlEnabled` 保证 ACC 能持续在控。
- `hasHistory` 表示是否存在可恢复的巡航状态。
- `lastDecision` 为恢复/历史逻辑保留最近有效动作。

5. 子模块二：ACC 巡航设定值管理

画布名称：`ACC巡航设定值管理`

源码类：

```
AccCruiseSettingStage
```

输入：

```
decision
```

输出：

```
timeGap, maxSpeed
```

内部状态：

状态	默认值	含义
-----	-----	-----
`initialized`	`false`	是否完成首次初始化
`timeGap`	`1.8 s`	当前目标时距
`maxSpeed`	`5.0 m/s`	当前最大设定速度

参数：

参数	默认值	含义
-----	-----	-----
`initialTimeGap`	`1.8 s`	初始时距
`minTimeGap`	`1.0 s`	最小时距
`maxTimeGap`	`3.0 s`	最大时距
`timeGapStep`	`0.2 s`	每次调整时距步长
`initialMaxSpeed`	`5.0 m/s`	初始最大速度
`minSpeed`	`0.0 m/s`	最小速度
`maxSpeedCap`	`5.0 m/s`	最大速度上限
`speedStep`	`1.3889 m/s`	每次调速步长，约等于 5 km/h

内部子模块：

子模块	对应源码	功能
-----	-----	-----
`ACC状态初始化`	`InitializeAccMemory`	首次运行时初始化时距和最大速度
`ACC下一周期时距计算`	`DetermineNextTimeGap`	根据 decision 调整时距并限幅
`ACC下一周期最大速度计算`	`DetermineNextMaxSpeed`	根据 decision 调整最大速度并限幅

5.1 `InitializeAccMemory`：状态初始化

功能：保证 `timeGap` 和 `maxSpeed` 在首次运行时有合法初值。

逻辑：

```
if initialized:
    timeGap = currentTimeGap
    maxSpeed = currentMaxSpeed
else:
    timeGap = clamp(initialTimeGap, minTimeGap, maxTimeGap)
    maxSpeed = clamp(initialMaxSpeed, minSpeed, maxSpeedCap)
    initializedNext = true
```

作用：

- 第一次进入时使用初始参数。
- 后续周期使用上一周期状态。
- 初始值如果越界，会先被限幅到合法范围。

5.2 `DetermineNextTimeGap`：下一周期时距计算

功能：根据决策编号更新目标时距。

逻辑：

```
if decision == 3:
    selectedTimeGap = currentTimeGap - timeGapStep
else if decision == 4:
    selectedTimeGap = currentTimeGap + timeGapStep
else:
    selectedTimeGap = currentTimeGap

timeGap = clamp(selectedTimeGap, minTimeGap, maxTimeGap)
```

作用：

- `decision = 3`：减小时距，车辆允许跟得更近。
- `decision = 4`：增大时距，车辆保持更大距离。
- 其他决策不改变时距。

5.3 `DetermineNextMaxSpeed`：下一周期最大速度计算

功能：根据决策编号更新巡航速度上限。

逻辑：

```
if decision == 1:
    selectedMaxSpeed = currentMaxSpeed - speedStep
else if decision == 2:
    selectedMaxSpeed = currentMaxSpeed + speedStep
else:
    selectedMaxSpeed = currentMaxSpeed

maxSpeed = clamp(selectedMaxSpeed, minSpeed, maxSpeedCap)
```

作用：

- `decision = 1`：降低巡航速度上限。
- `decision = 2`：提高巡航速度上限。
- 其他决策不改变速度上限。

6. 子模块三：ACC 目标速度门控计算

画布名称：`ACC目标速度门控计算`

源码类：

AccTargetSpeedStage

输入：

egoSpeed, leadSpeed, leadDistance, timeGap, maxSpeed, enable

输出：

```
targetSpeed
```

参数：

参数	默认值	含义
`minDistance`	`5.0 m`	最小跟车距离
`distanceGain`	`0.25`	车距误差增益
`speedGain`	`0.4`	相对速度增益

内部子模块：

子模块	对应源码	功能
`ACC目标速度核心计算`	`CalcAccTargetSpeed`	根据时距模型、距离误差、相对速度计算目标速度

门控逻辑：

```
targetSpeed = enable ? calcTargetSpeed : 0.0
```

作用：

- ACC 使能时输出计算得到的目标速度。
- ACC 未使能时输出 0，避免继续向纵向控制发送跟车目标。

6.1 `CalcAccTargetSpeed`：目标速度核心计算

功能：由前车速度、车距误差和相对速度计算目标速度。

步骤一：前车距离安全处理

```
safeLeadDistance = max(leadDistance, 0)
```

作用：如果输入距离异常为负数，先按 0 处理。

步骤二：期望跟车距离

```
timeGapDistance = egoSpeed * timeGap  
desiredDistance = max(minDistance, timeGapDistance)
```

作用：

- 车速越高，按时距模型要求更大的跟车距离。
- 低速时至少保留 `minDistance`。

步骤三：车距误差

```
distanceError = safeLeadDistance - desiredDistance
```

含义：

- ``distanceError > 0``：实际距离大于期望距离，可以适当提速。
- ``distanceError < 0``：实际距离小于期望距离，需要减速。

步骤四：相对速度

```
relativeSpeed = leadSpeed - egoSpeed
```

含义：

- ``relativeSpeed > 0``：前车比自车快，可以补速。
- ``relativeSpeed < 0``：前车比自车慢，需要提前减速。

步骤五：无限幅目标速度

```
rawTargetSpeed =  
  leadSpeed  
  + distanceGain * distanceError  
  + speedGain * relativeSpeed
```

作用：

- 以前车速度为基准。
- 用车距误差修正跟车距离。
- 用相对速度修正速度匹配。

步骤六：目标速度限幅

```
targetSpeed = clamp(rawTargetSpeed, 0, maxSpeed)
```

作用：

- 不允许目标速度为负。
- 不允许超过当前巡航速度上限。

7. 画布整体数据流

```

acc_input
├── frame_id -----> acc_output.frame_id
├── ego_speed_mps -----> AccTargetSpeed.egoSpeed
├── lead_speed_mps -----> AccTargetSpeed.leadSpeed
├── lead_distance_m -----> AccTargetSpeed.leadDistance
└── command_type -----> AccTargetSpeed.commandType

```

```

AccTargetSpeed
├── AccDecisionStage
│   ├── DetermineAccSystemState
│   ├── EvaluateAccDecision
│   ├── DetermineAccEnable
│   └── SelectAccLastDecision
├── AccCruiseSettingStage
│   ├── InitializeAccMemory
│   ├── DetermineNextTimeGap
│   └── DetermineNextMaxSpeed
└── AccTargetSpeedStage
    └── CalcAccTargetSpeed

```

```

AccTargetSpeed 输出
├── targetSpeed -> acc_output.target_speed_mps
├── enable -----> acc_output.enable
├── valid -----> acc_output.valid
├── timeGap -----> acc_output.time_gap_s
├── maxSpeed ----> acc_output.max_speed_mps
├── decision ----> acc_output.decision
└── systemState -> acc_output.system_state

```

8. 汇报时可以这样讲

‘demo1’的核心不是单个“目标速度公式”，而是一个完整 ACC 状态机加目标速度计算链路。

第一层是 Pangu 模块输入输出，负责把 ‘AccInput’ 消息送入算法、把 ‘AccOutput’ 发布出去。

第二层是 ‘AccTargetSpeed’ 算法入口，它把 ACC 拆成三个阶段：先决策、再更新巡航设定值、最后计算目标速度。

第三层中，‘AccDecisionStage’ 负责判断 ACC 当前是否在控、驾驶员指令应该产生什么决策、下一周期是否继续在控；‘AccCruiseSettingStage’ 负责把决策转换为时距和限速参数；‘AccTargetSpeedStage’ 负责根据前车速度、车距误差和相对速度计算目标速度，并在 ACC 未使能时把目标速度关断。

最终输出的 ‘targetSpeed’ 是纵向控制要跟踪的目标速度；‘enable’ 表示 ACC 是否接管；‘decision’ 和 ‘systemState’ 是用于调试和观测的状态量。

9. 补充：为什么 ACC 目标速度核心计算看起来很复杂

‘ACC目标速度核心计算 / CalcAccTargetSpeed’ 在源码里只有一个 ‘run()’，但它不是单纯把一个公式直接输出，而是把跟车控制拆成了多段安全计算。GAASD 扫描后会每一个比较、乘法、加法、三目选择、限幅分支都展开成画布节点，所以在画布上会显得比公式本身复杂。

9.1 这块实际做了哪几件事

从功能上看，`CalcAccTargetSpeed` 分为六步：

```
leadDistance 保护
-> 期望跟车距离 desiredDistance
-> 车距误差 distanceError
-> 相对速度 relativeSpeed
-> 未限幅目标速度 rawTargetSpeed
-> 目标速度限幅 targetSpeed
```

每一步都有明确的物理含义：

```
| 步骤 | 中间量 | 物理含义 | 为什么需要 |
| --- | --- | --- | --- |
| 1 | `safeLeadDistance` | 可用前车距离 | 防止上游传入负距离导致控制异常 |
| 2 | `desiredDistance` | 期望跟车距离 | 把时距策略转换成距离目标 |
| 3 | `distanceError` | 实际距离与期望距离的偏差 | 判断当前距离是偏大还是偏小 |
| 4 | `relativeSpeed` | 前车速度减自车速度 | 判断两车速度是否匹配 |
| 5 | `rawTargetSpeed` | 未限幅目标速度 | 综合前车速度、距离误差、相对速度得到控制目标 |
| 6 | `targetSpeed` | 最终目标速度 | 限制在 `[0, maxSpeed]` 合法范围内 |
```

9.2 源码与画布节点的对应关系

源码中的核心逻辑如下：

```
safeLeadDistance = leadDistance < 0 ? 0 : leadDistance;

timeGapDistance = egoSpeed * timeGap;
desiredDistance = max(minDistance, timeGapDistance);

distanceError = safeLeadDistance - desiredDistance;
relativeSpeed = leadSpeed - egoSpeed;

rawTargetSpeed =
  leadSpeed
  + distanceGain * distanceError
  + speedGain * relativeSpeed;

targetSpeed = clamp(rawTargetSpeed, 0, maxSpeed);
```

扫描到画布后通常会变成如下节点组合：

```
| 源码语句 | 画布上会展开成 |
| --- | --- |
| `leadDistance < 0 ? 0 : leadDistance` | `less-than` + 条件分支/三目选择 + 合并 |
| `egoSpeed * timeGap` | 一个乘法节点 |
| `max(minDistance, timeGapDistance)` | 比较节点 + 选择节点 |
| `safeLeadDistance - desiredDistance` | 一个减法节点 |
| `leadSpeed - egoSpeed` | 一个减法节点 |
| `distanceGain * distanceError` | 一个乘法节点 |
| `speedGain * relativeSpeed` | 一个乘法节点 |
| 三项相加 | 两个加法节点串联 |
| `clamp(rawTargetSpeed, 0, maxSpeed)` | 小于 0 判断、大于 maxSpeed 判断、三个输出分支、合并节点 |
```


因此，画布复杂不是因为算法在做很多“额外动作”，而是因为 GAASD 把一行表达式拆成了数据流图。

9.3 每个中间量对目标速度的影响

‘safeLeadDistance’

```
safeLeadDistance = max(leadDistance, 0)
```

‘leadDistance’ 来自前车检测或仿真接口。理论上它应该大于等于 0，但接口异常、无目标、初始化瞬间都可能出现非法值。这里把负数压成 0，是为了让后续距离误差不会出现物理上不成立的“负距离”。

‘desiredDistance’

```
timeGapDistance = egoSpeed * timeGap  
desiredDistance = max(minDistance, timeGapDistance)
```

这是时距控制的核心。车辆速度越高，安全跟车距离应该越大；但低速或起步时，‘egoSpeed * timeGap’可能很小，所以还要保留一个最小距离 ‘minDistance’。最终的期望距离取二者较大值。

‘distanceError’

```
distanceError = safeLeadDistance - desiredDistance
```

‘distanceError’ 表示当前车距是否满足目标：

‘distanceError’	含义	对目标速度的影响
大于 0	实际车距大于期望车距	可以适当加速靠近
等于 0	实际车距接近期望车距	保持跟车
小于 0	实际车距小于期望车距	需要减速拉开距离

‘relativeSpeed’

```
relativeSpeed = leadSpeed - egoSpeed
```

这个量用于速度匹配：

‘relativeSpeed’	含义	对目标速度的影响
大于 0	前车比自车快	允许目标速度提高
等于 0	两车速度一致	速度项不修正
小于 0	前车比自车慢	提前降低目标速度

如果只看距离误差，车辆可能反应滞后；加入相对速度后，ACC 可以提前感知“正在接近”或“正在拉远”的趋势。

‘rawTargetSpeed’

```
rawTargetSpeed =  
  leadSpeed  
  + distanceGain * distanceError  
  + speedGain * relativeSpeed
```

这条公式可以理解成三部分叠加：

部分	作用
--- ---	
`leadSpeed`	以前车速度作为基本跟随速度
`distanceGain * distanceError`	根据车距偏差修正速度
`speedGain * relativeSpeed`	根据相对速度趋势修正速度

所以这不是简单“目标速度等于前车速度”，而是“以前车速度为基准，再用距离误差和相对速度做闭环修正”。

`targetSpeed`

```
targetSpeed = clamp(rawTargetSpeed, 0, maxSpeed)
```

最终目标速度必须满足两个边界：

边界	作用
--- ---	
下限 0	不允许输出负目标速度
上限 `maxSpeed`	不允许超过当前巡航速度上限

这一步也是画布复杂的主要来源之一，因为 `clamp` 在扫描后会变成两个比较条件和三个分支输出。

9.4 这块与 ACC 其他模块的关系

`CalcAccTargetSpeed` 本身不决定 ACC 是否启用，也不决定 `timeGap` 和 `maxSpeed` 的更新。它只消费前面模块给出的结果：

```
AccDecisionStage  
-> enable / decision / systemState  
  
AccCruiseSettingStage  
-> timeGap / maxSpeed  
  
CalcAccTargetSpeed  
-> targetSpeed
```

也就是说：

- `decision` 决定是否调整时距或限速。
- `timeGap` 决定期望距离。
- `maxSpeed` 决定目标速度上限。
- `enable` 决定 `AccTargetSpeedStage` 是否把 `targetSpeed` 输出给外部。

这也是为什么画布中目标速度计算看起来和决策、状态、参数更新紧密相连；它不是孤立计算，而是 ACC 闭环的一段。

9.5 汇报时可以这样解释这块复杂度

可以按下面这段话讲：

“ACC 目标速度核心计算”并不是单纯输出前车速度，而是把跟车问题拆成“目标距离”和“速度匹配”两个闭环。首先根据自车速度和目标时距计算期望车距，再用实际前车距离减去期望车距得到距离误差；同时用前车速度减去自车速度得到相对速度。最终目标速度以前车速度为基准，叠加距离误差修正项和相对速度修正项，最后再通过上下限保护得到可输出的目标速度。画布上节点较多，是因为源码里的 `max`、`clamp`、条件选择和多项加法都被 GAASD 展开成了基础数据流节点。